# Empirical Comparison of CNN Architectures on CIFAR-10.

**Vishnu Nandurkar**

**Abstract:**

In this paper, an analysis of the performance of different CNN architectures on the CIFAR – 10 dataset is presented. Three CNN architectures were used for comparison – VGG, ResNet and simple CNN. For each of the architectures, various hyperparameters were tuned to achieve maximum performance. The reported results include the training, testing and validation accuracies along with the trend of losses with respect to number of epochs the model has run.

## Introduction:

Convolutional Neural Networks have been monumental in image classification tasks in the past decade. Different variations in CNN architectures have led to a better understanding of how information can be processed from images efficiently so as to not lose any image features and shorten the runtime (as image classification tasks require the model to go through thousands of images).

Residual networks (ResNet) have been particularly good at this by being able to train very deep models by using 'skip connections' to skip layers in order to mitigate the vanishing gradient problem. In this paper as well, this architecture has performed the best in the classification task of the CIFAR – 10 dataset. Resnet is a very complex architecture, and it was prudent to contrast it with a simpler yet competent one. Namely, the Visual Geometry Group (VGG) network which stacks small (3x3) convolutional filters that help increase depth and non – linearity was used. Such filters increase the network's ability to classify (by enhancing feature extraction) while reducing parameters. It also follows these filters with 2x2 max pooling layers to reduce spatial dimension resulting in a uniform architecture. Lastly, a simple CNN architecture which is a lightweight network consisting of 2 convolutional layers (size = 5) each followed by an activation function and pooling layers for feature extraction.

In this paper, the well-known CIFAR-10 dataset has been used. This is a multiclass image classification task of 32x32 pixel color images into 10 classes. In this task as expected ResNet performed the best followed by VGG and simple CNN. But this paper will also attempt to explain this result with respect to the specific hyperparameter values used for each network to maximize performance.

## Methodology:

The CIFAR – 10 dataset's training size was halved to run the models faster. The dataset was too large to work with given the computational resources available. Additionally, all the networks were run for 65 epochs (instead of the standard >100) as the dense models were taking time to run. Nevertheless, the accuracy results obtained by training for 65 epochs is sufficient to analyzing and estimating model performance as for higher epoch values though the accuracy was higher the rate of learning was low. This makes easier to predict the actual best performance of the network.

ResNet: Every basic residual block consisted of 2 convolutional layers and a skip connection that adds the input to output of the second convolutional layer. There is optional batch normalization available which can

be used based on configuration. The first convolutional layer is 3x3 with stride = 1 and is identical to the second one. ReLU activation was used (can be manually changed via command prompt). Now, the full network stacks multiple basic residual blocks in three stages to increase filters and reduce spatial size. First there is an initial 3x3 convolutional layer that transforms the input RGB image (32x32) into 16 feature maps without down sampling. Then an activation is applied. Each residual stage is made up of n basic residual blocks. There are 3 such layers/stages. After each layer the feature map size reduces as number of filters increase. This is followed by adaptive average pooling which reduces each feature map to a single value which produces a 64-dimensional vector. Finally, at the end there is simple linear classifier.

VGG: The network consists of a convolutional feature extractor and a fully connected classifier. The extractor stacks multiple convolutional layers followed by optional batch normalization and an activation function. After every two convolutional layers there is a pooling layer to reduce dimension. The feature extractor starts with 32 filters that doubles after every two layers and goes up to 256. Kernal size is 3x3 with padding = 1 and stride = 1. The pooling type, depth (number of convolutional layers) and activation function are all configurable. As the input is 32x32 and each pooling halves the size, the pooling operations are depth // 2. The final feature maps (outputted as a single tensor) are of size $32/(2^{(\text{depth } // \ 2)})$. The classifier flattens the feature maps into a vector of size out_channels x feature_size x feature_size (as feature maps are squares). Here out_channels refers to the number of convolutional filters in the last year. Then this vector is passed through 2 linear layers (512 units) with activation function and dropout (rate is configurable) after each layer. Lastly, a final linear layer with num_classses outputs (10 as its CIFAR – 10).

Simple CNN: The feature extractor for this built as a Sequential container. The first convolutional layers input 3 channels (RGB) and outputs 6 feature maps. This is followed by padding, optional batch normalization, activation function and a pooling layer. All of this is configurable. The final pooling reduces the size from 32x32 to 16x16. The second convolutional layer inputs 6 channels and outputs 16 feature maps. As this layer has no padding, the spatial size shrinks from 16x16 to 12x12. Then like before, this is followed by an activation and pooling layer which reduces size to 6x6. So, the output of the feature extractor is a tensor with 16 feature maps each of size 6x6. This is followed by the classifier which is a Sequential block that flattens input into a vector. This vector goes through two fully connected linear (FC1 = Linear(576, 120), FC2 = Linear(120, 84)) layers each of which is followed by an activation and dropout. At the end, a final linear layer (Linear(84, 10)) returns the logits for the CIFAR – 10 classes.

Hyperparameters: Listed below are all the configurations for the models along with available options for each configuration. The models used in the next section are a result of a combination of them.

1. Model Architecture: Options include simple CNN, VGG and ResNet. Default architecture used was the simple CNN.
2. Depth: Specifies number of Convolutional layers for VGG and number of blocks for ResNet. The default number is set to 2.
3. Activation function: Options include ReLU, sigmoid and tanh. The default activation is ReLU.
4. Pooling: Options were max, average and stochastic pooling. Max pooling layer was the default.
5. Optimizer: Options included stochastic gradient descent (SGD), Adam optimizer and root mean square propagation (RMSprop). Default optimizer was SGD.
6. Learning rate: Determines the step size when adjusting weights of the network during optimization. Default rate was 0.01.

7. Momentum: Helps the SGD optimizer to achieve faster convergence. Default momentum was the widely used standard 0.09.
8. Weight decay: Helps penalize large weight values so that the model learns smoother and simpler functions. Default weight decay was 0.0005.
9. Batch size: Default batch size was 128.
10. Step size: Helps specify the interval after which learning rate needs to be adjusted. Default step size is 20.
11. Gamma: It defines the rate of decrease of learning rate to help model converge more efficiently. Default gamma value is 0.1.
12. Dropout: It's the dropout probability. Default value is 0.
13. Batch normalization: Normalizes input activations of each layer to speed up training and convergence. Its optional feature which can be added to any of architectures.

Three models (one for each CNN architecture) were made using specific combinations of hyperparameters by passing them as arguments. These combinations are designed to extract the best performance from each of CNN architectures. The three models along with their hyperparameters are as listed below.

1. ResNet model: depth = 56, optimizer = SGD, learning rate = 0.1, momentum = 0.9, weight decay = 0.0005, batch size = 64, epochs = 65, step size = 30, gamma = 0.2, activation function = ReLU and dropout = 0.3. Batch normalization is also included in this model.
2. VGG model: depth 16, optimizer = SGD, learning rate = 0.1, momentum = 0.9, weight decay = 0.0005, batch size = 64, epochs = 65, step size = 30, gamma = 0.2, activation = ReLU and dropout = 0.3. Batch normalization is also included in this model.
3. Simple CNN model: optimizer = RMSprop, learning rate = 0.001, weight decay = 0.0002, batch size = 64, epoch = 65, step size = 25, gamma = 0.2, activation = leaky ReLU, dropout = 0.25. Batch normalization is also included in this model.

## **Results and Analysis:**

| Network | Validation Acc | Testing Acc | Training Acc |
|---|---|---|---|
| ResNet | **88.88%** | **86.5%** | **95.06%** |
| VGG | 87.12% | 85.71% | 94.32% |
| Simple CNN | 66.36% | 69.23% | 65.72% |

The training accuracies provided above are ones obtained from the best model. The best model is the one with highest validation accuracy after 65 epochs. The loss curves for validation and training accuracy for all three models have been plotted in the appendix. ResNet achieves the best performance followed closely by VGG. Simple CNN, likely due to lack of depth, performs the worst. The gap between training and testing accuracy indicates overfitting in VGG and ResNet.

In the loss curves (shown in appendix) of ResNet, there is a drop around epoch 30 which likely corresponds to the first learning rate reduction (step size = 30). The high training accuracy (95.06%) and moderately well validation accuracy (88.88%) suggest the model has the capacity to memorize the dataset and also can generalize well enough to achieve decent performance. For VGG, validation loss declines more slowly than training loss and stabilizes around 0.40 – 0.55 after epoch 40. The learning rate of 0.1 causes noisy validation loss until the first step decay at epoch 30 after which its decrease is more stable. There is an 8.6%

gap between training and validation accuracy which indicate overfitting but is less severe than ResNet. This is likely because VGG has fewer parameters than ResNet. VGG's validation accuracy continues to improve till the end of 65 epochs which indicates it has the potential to do better with more epochs. Compared to these models, the simple CNN's validation loss remains high and fluctuates between 0.93 and 1.05 after epoch 30, showing little to no sign of improvement. Although there is no apparent overfitting, accuracies remain low which shows that this shallow architecture fails to capture the complex patterns of the CIFAR – 10 dataset.

In summary, ResNet outperforms VGG in testing accuracy by approximately 3.5% owing to its skip connections feature without which optimization becomes harder with increase in depth for models like VGG. However, with proper regularization VGG performs pretty well compared to ResNet and as discussed earlier with more epochs it had potential to do even better.

**Conclusion:**

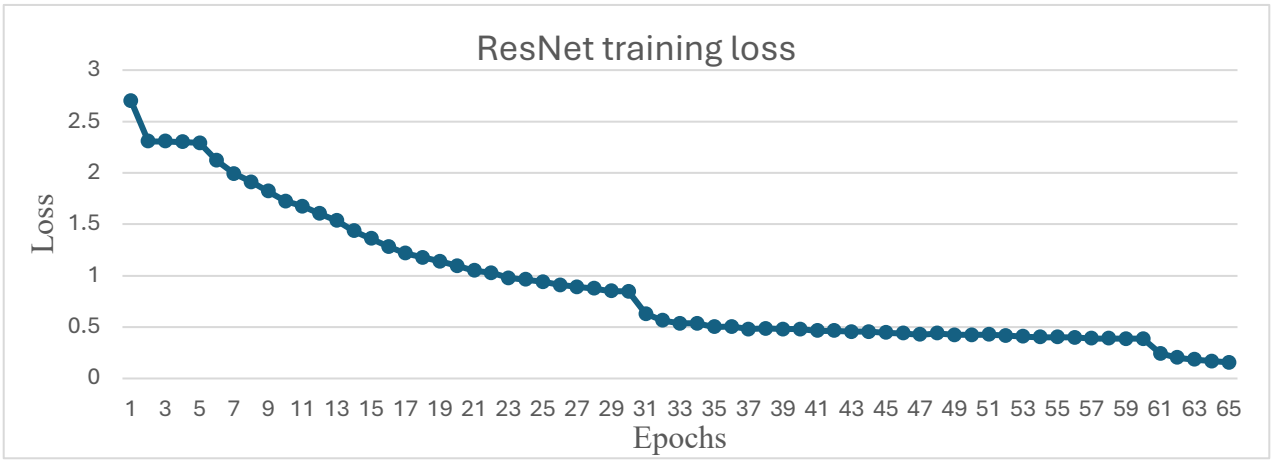
The results of this paper clearly demonstrate that deeper architectures with residual connections are essential for capture complex visual patterns found in datasets like CIFAR-10, even if only half the dataset is being used. A higher dropout (0.3) for ResNet and VGG was set to mitigate overfitting, but it seems it was still not high enough. This means these models could have benefited from a stronger regularization. A simple CNN architecture despite its speed and lightweight structure is inferior to deeper networks. These findings align with established literature and highlight the importance of a deeper network choice for image classification tasks.

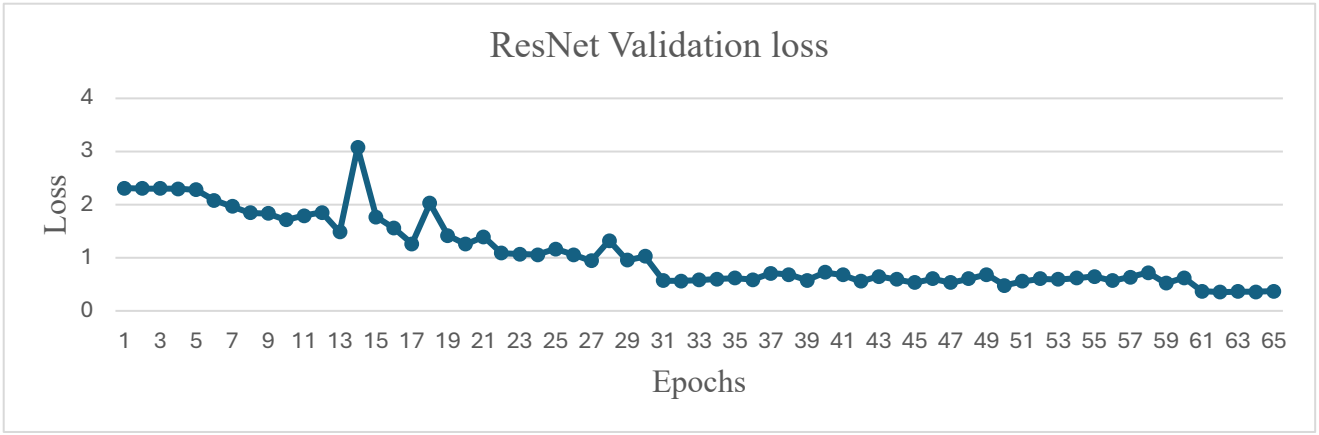
**References:**

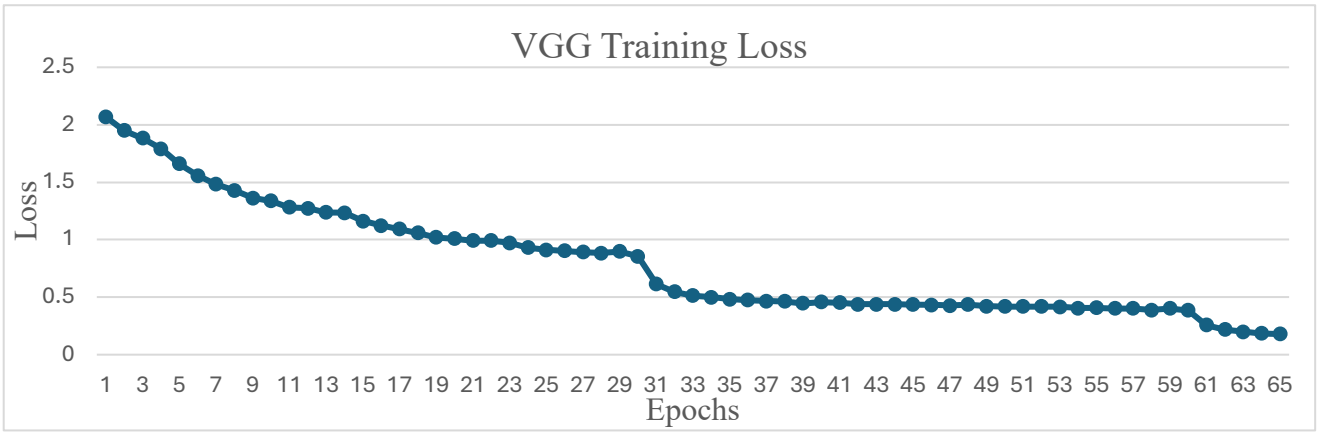

1. Learning Multiple Layers of Features from Tiny Images, Alex Krizhevsky, Technical Report, Computer Science Department, University of Toronto, 2009.
2. Sangwook Lee "Deep Learning Hyperparameter Optimization Using Evolutionary Computation Techniques" Journal of Internet of Things and Convergence 11.5 9 (2025) : 9.

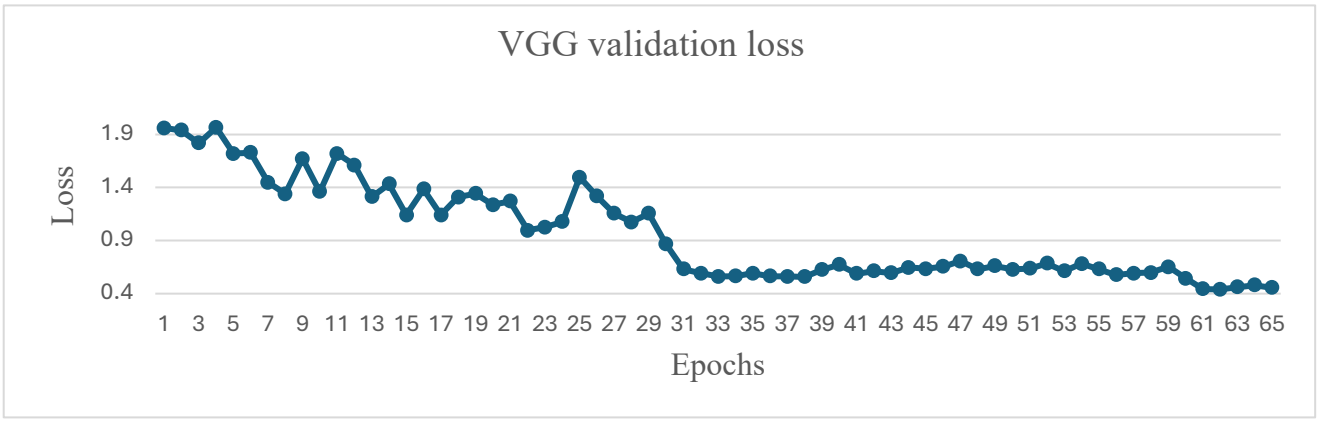

**Appendix:**

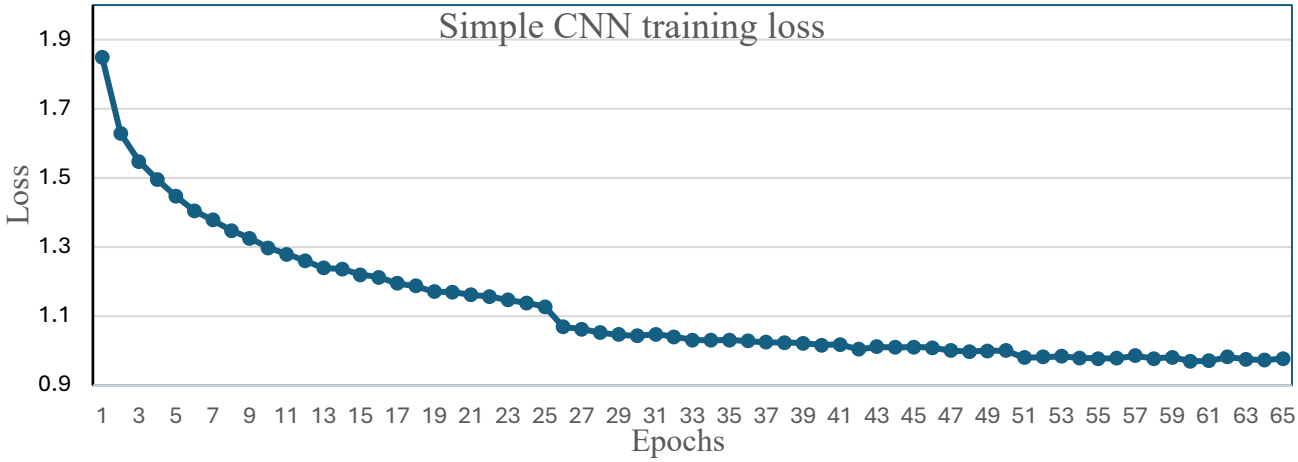
The variation in validation and training loss with respect to epochs for each model:

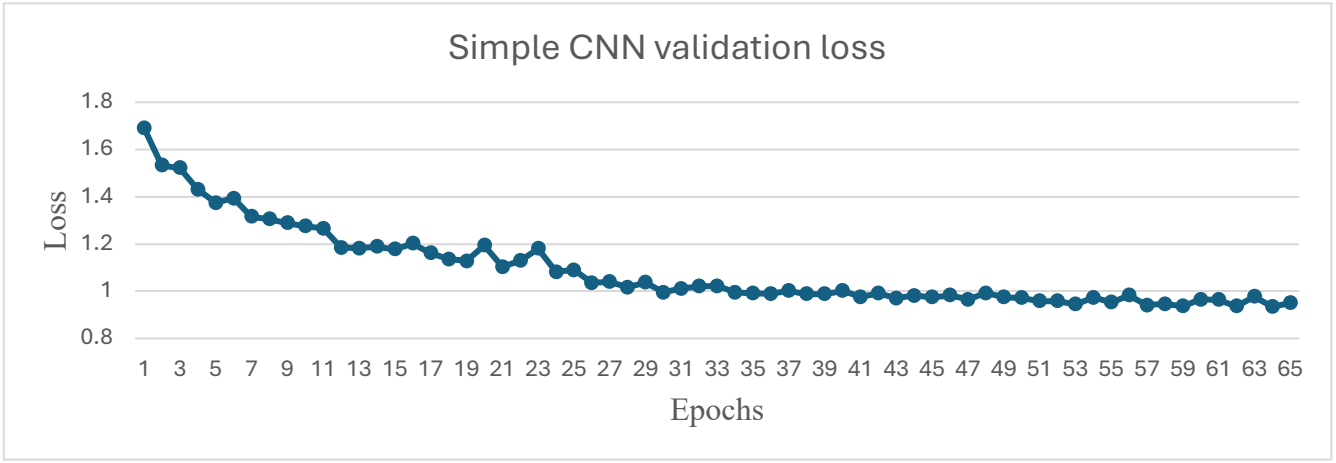



ResNet Validation loss
Loss
4
3
2
1
0
1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41 43 45 47 49 51 53 55 57 59 61 63 65
Epochs


VGG Training Loss
Loss
2.5
2
1.5
1
0.5
0
1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41 43 45 47 49 51 53 55 57 59 61 63 65
Epochs


VGG validation loss
Loss
1.9
1.4
0.9
0.4
1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41 43 45 47 49 51 53 55 57 59 61 63 65
Epochs


Simple CNN training loss
Loss
1.9
1.7
1.5
1.3
1.1
0.9
1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41 43 45 47 49 51 53 55 57 59 61 63 65
Epochs


Simple CNN validation loss
Loss
1.8
1.6
1.4
1.2
1
0.8
1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41 43 45 47 49 51 53 55 57 59 61 63 65
Epochs